2026

# Major Project Proposal

FIGMA-TO-CODE GENERATOR PLUGIN

MATTHEW DENT (STUDENT)

# Contents

Introduction ..... 2
Aim & Objectives ..... 2
Requirements, Specification & Feasibility ..... 3
Technology & Research ..... 5
Project Management ..... 8
Initial Design & Outcome ..... 9

# Figma to Tailwind Code Generator Plugin

## Introduction

Frontend development relies on communication between designers and developers. Tools such as Figma have become standard for the UI and interaction design industries, while frameworks like React and Tailwind CSS thrive in component-based development environments. A gap persists between designs and readily produced code, developers are required to recreate designs from Figma to HTML, CSS, and React components. This adds time to the process and maintains the probability of creating errors.

This project develops a Figma plugin that converts the design in a selected Figma frame and converts it into HTML and React components styled through Tailwind utility classes. Using a UI panel the extracted properties are editable before code production for developers to maintain control over the design. All functionality is processed in the Figma application, which allows developers to inspect and edit the values without redirecting to external resources.

By automatically converting the node properties into Tailwind classes, the plugin aims to increase efficiency in design-to-code production and improve component-based workflows. Existing plugins generally rely on inline CSS, arbitrary values or they lose fidelity of complex functions. This plugin focuses on accuracy and developer control within the application itself.

This project crosses between interaction design and development tooling. The project is formed from areas such as Frontend Development; Design-to-Code automation; and Developer tooling. It addresses issues faced by frontend developers, full-stack developers, students and more.

## Aim & Objectives

The aim of this project is to design and implement a Figma plugin that converts a selected frame / group into an editable, readily produced React + Tailwind component code, with accurate mapping of node properties, all performed inside Figma to assist in increasing efficiency in component-based design workflows.

Specific Objectives:

1. Implement functionality to extract  node properties from selected Figma frames using the Figma Plugin API.

2. Develop a mapping layer to convert Figma node values into Tailwind utility classes whilst avoiding arbitrary values.
3. Produce accurate and stable HTML with Tailwind CSS and React + Tailwind components.
4. Design a plugin user interface that displays extracted property values for developers to edit and preview before producing generated code.
5. Conduct testing of the functional and usability states. Ensuring the plugin responds as expected, and further refine the UI and code based on feedback.

These objectives are achievable within the implementation time and relevant for frontend workflows.

## Requirements, Specification & Feasibility

Backlog:

| Priority | Feature | Description |
|---|---|---|
| Must | Node Extraction | Extract layouts, auto-layouts, nested components, spacing, sizing, typography, and colours from Figma frames. |
| Must | Mapping Logic | Convert Figma node values into Tailwind utility classes using scales, handle arbitrary values. |
| Must | Code Generation | Generate HTML and React + Tailwind components. |
| Must | Plugin UI Panel | Display extracted values for developer preview and editing |
| Should | React Component Structuring | Apply component hierarchy in output for immediate integration |
| Should | Preview & Copy | Provide live preview of generated code, allow copy to clipboard |
| Could | Tailwind Theme Tokens | Allow users to export values using Tailwind custom themes. |
| Could | Code Export | Enable downloadable export of generated code. |

Technical Feasibility:

This section focuses on the technologies selected for this project and the workflow required to complete the implementation of the code generator plugin. The project uses a combination of design tools and frontend frameworks to maintain clear and accurate code production.

Figma Plugin API  is central to this project as it accesses the elements within the Figma application, providing access to the node properties including position, size, spacing, typography and colours. The API assists in extracting complex features such as layout directions and nested auto layouts. By implementing the plugin in the Figma application users can inspect the extracted values and generated code without exiting the design environment.

TypeScript is implemented to maintain the plugin logic. It improves the reliability of the code as extracting node properties contains a variety of types, which TypeScript can manage reliably, this reduces the chances of errors during running and extraction.

Tailwind's utility-first framework is selected for styling the generated code, the plugin converts Figma measurements, colours, spacing and typography into Tailwind utility classes. It follows a generic tailwind scale measurement to ensure accurate representation of the Figma design in the generated code output. Tailwind ensures consistency within the styling and negates the chance of arbitrary values. The plugin maps the layout flow and style properties into Tailwind's responsive utilities.

Node.js provides the environment for building and generating code. TypeScript, HTML and React are supported for generating code and enabling the plugins interactivity.

Economic Feasibility:

The project requires no financial assistance as the platforms used are free of usage and open source. Fees will be used for researching other existing plugins to review.

Operational Feasibility:

The plugin will operate entirely in the Figma application, which reduces the need for a server or deployment. Developers only require access to a Figma account, ensuring easier use of the plugin.

Initial Test Plan:

Unit Testing: Validate mapping functions for converting pixel values to Tailwind utility classes.

Functional Testing: Ensure extraction and code generation is equivalent to the design and produces valid outputs, including nested frames and auto-layouts.

Usability Testing: Conduct informal testing with frontend users to test usability of the plugin and interface.

Edge Case Testing: Test layouts, nested layouts, auto-layouts, colour themes and differentiating designs.

## Technology & Research

Core Technologies:

The core technologies implemented into this project are the Figma Plugin API, TypeScript, Tailwind CSS and React. These technologies were chosen for their positive contribution towards the goals created for the plugin. To extract Figma properties and convert it into Tailwind styled HTML and React components, whilst allowing developers to maintain control over editing through the interface.

Justification:

Figma Plugin API:

The Figma Plugin API provides access to the node properties within the Figma application. The plugin can inspect the frame to view the layout, spacing, colour, sizing and typography values through the user interface. The API allows developers to preview and modify properties within Figma to ensure consistency with the original design.

TypeScript:

TypeScript is used as the development language for creating the plugin and type checking the extracted properties of the Figma nodes. The benefit of TypeScript is its management of types on extraction of node properties as they vary widely within the frame and layouts. It ensures the conversion of properties from the node to Tailwind is smooth and executes scripts for processing plugin functionality.

Tailwind CSS:

Tailwind CSS is selected as the styling Framework for its consistency for generating code across the plugin. Through mapping Figma node values into Tailwind utility classes, the plugin follows Tailwind scales to minimize arbitrary values being outputted. Tailwinds system follows a modular scheme which assists in creating modular components for frontend development.

React:

React framework was chosen for generating components from Figma extracted node properties. Its component-based architecture assists Tailwind in creating modular components together allowing developers to reuse UI elements. React combined with Tailwind allows for the plugin output to be consistently designed and scalable for further development.

Research & Background:

Research for this project is directed at design-to-code automation, frontend development workflows and the difficulties with consistent fidelity across design and implementation processes. Through studies on design-to-code tools, challenges became apparent with translating Figma designs into code manually, guiding the creation of the plugin's requirements. Open-source Figma plugins have been reviewed to understand the approach to extracting, converting and parsing node properties into utility-first CSS.

To gather information a survey was conducted, respondents ranging from computing students to professionals in the development roles. This assisted in identifying key views and aspects of the functionality required for the plugin to oppose the recorded difficulties with the current state of design-to-code implementation. Additionally, the Figma inspect panel was considered inconsistent with developers, which insinuates visual comparison is prominent whilst developing UI conversions. This research supports the development decisions chosen for an interactive in-app interface instead of solely automated exportations.

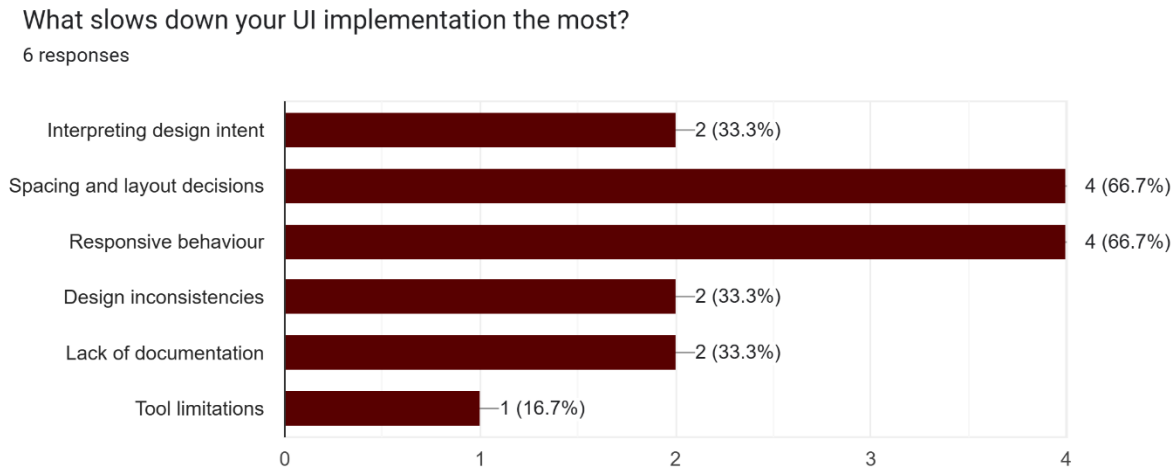


*Figure 1 - Bottlenecks in UI implementation*

Styling practices with Tailwind CSS were prominent amongst respondents as a primary choice, due to the consistency and scalability of the framework. Developers stated difficulties with the handling of arbitrary spacing values from design transfers as displayed in Figure 1 above, resulting in manual adjustment of the code. This information supported the necessity for mapping Tailwind values through accurate scaling to reduce workload after code conversion.

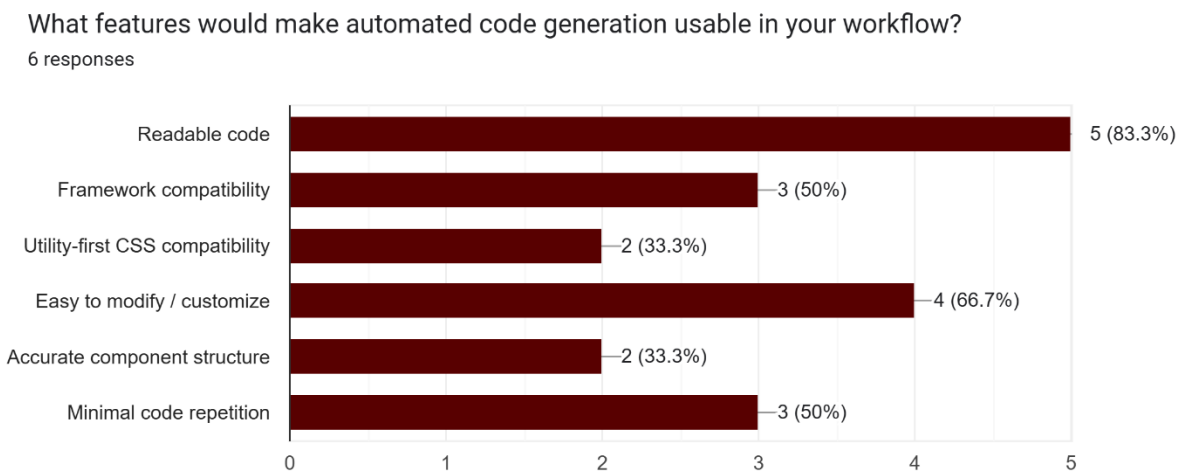


*Figure 2 - Prominent Features UI implementation*

The research shown in Figure 2 demonstrates the importance of clear UX/UI, which ensures the necessity of an interface for previewing the extracted properties. The plugin works completely in-app to reduce the need for external resources and increase workflow efficiency. The interactive panel allows developers to control the parsed data to reduce design errors before code generation.

# Project Management

Methodology:

The project will follow an Agile iterative methodology which is divided into sprints that revise the current objectives and requirements of the project, document the activities performed and validate the future implementation plan and refinement of code. GitHub will be used for Version control, branching and tracking bug fixes and persistent issues. Miro will visualise documentation and sprints.

Initial Sprint Plan:

Sprint 1: Project Setup

Establish a technical basis of the project and define the structure of the design-to-code plugin.

- Create the Figma plugin using TypeScript, configuration of the manifest.json, and developing the pipeline environment.
- Create a GitHub repository and branch the development stages.
- Implement esbuild to ensure Figma plugin building and connecting established.
- Review Figma node properties and hierarchy system to understand how to extract properties of the node.
- Define the models for node data extraction and assure it is TypeScript formatted.
- Review open-source plugins and repositories to understand processes for parsing, extracting and inspecting node properties.

Deliverables include a working plugin basis, documentation process of extracting and reading node properties, and researching property extraction and parsing.

Sprint 2: Design Extraction and Tailwind Mapping

Implement extraction of Figma designs and converting into Tailwind utility classes.

- Develop logic for extracting frames, groups, auto layouts and nested components.
- Extract layout properties, flex direction, alignment, spacing, padding, width, height and positioning.
- Extract typography, styling properties, font size, font weight, line height, letter spacing, colour and background.
- Implement mapping to convert extracted node properties into Tailwind CSS.
- Create Tailwind CSS scales.

Deliverables include node extraction of auto layouts, tailwind utility mapping for stylings, producing HTML inline Tailwind CSS code generation.

Sprint 3: UI panel and Component Generation

Enable user interactive panel, previewing and editing of generated code in Figma application.

- Develop plugin UI panel using HTML and CSS and JavaScript to display extracted node properties.
- Display editable fields for node property values.
- Implement component generation with React and Tailwind CSS.
- Implement code preview for HTML and React outputs.
- Allow for copying of code generation outputs.

Deliverables include an interactive plugin UI panel, editable value fields for development, React + Tailwind component generation, and HTML and React code generation outputs.

Supervisor Plan:

Weekly meetings with the assigned supervisor will be conducted to review current state of the project as well as discuss issues and further development steps to be taken. Meetings will be documented with a Discussion, Decisions, Actions plan to reflect on the meeting and understand the outcome of the discussion.

## Initial Design & Outcome

Early UI concepts:

Initial design concepts focused on developing a frontend interface for viewing extracted data from the Figma nodes to ensure extraction code gathers all relevant information. This process was further altered due to feedback from a survey requesting developer control of extracted data. The UI panel was redesigned to include a preview of current state of code generation and that value fields are editable.

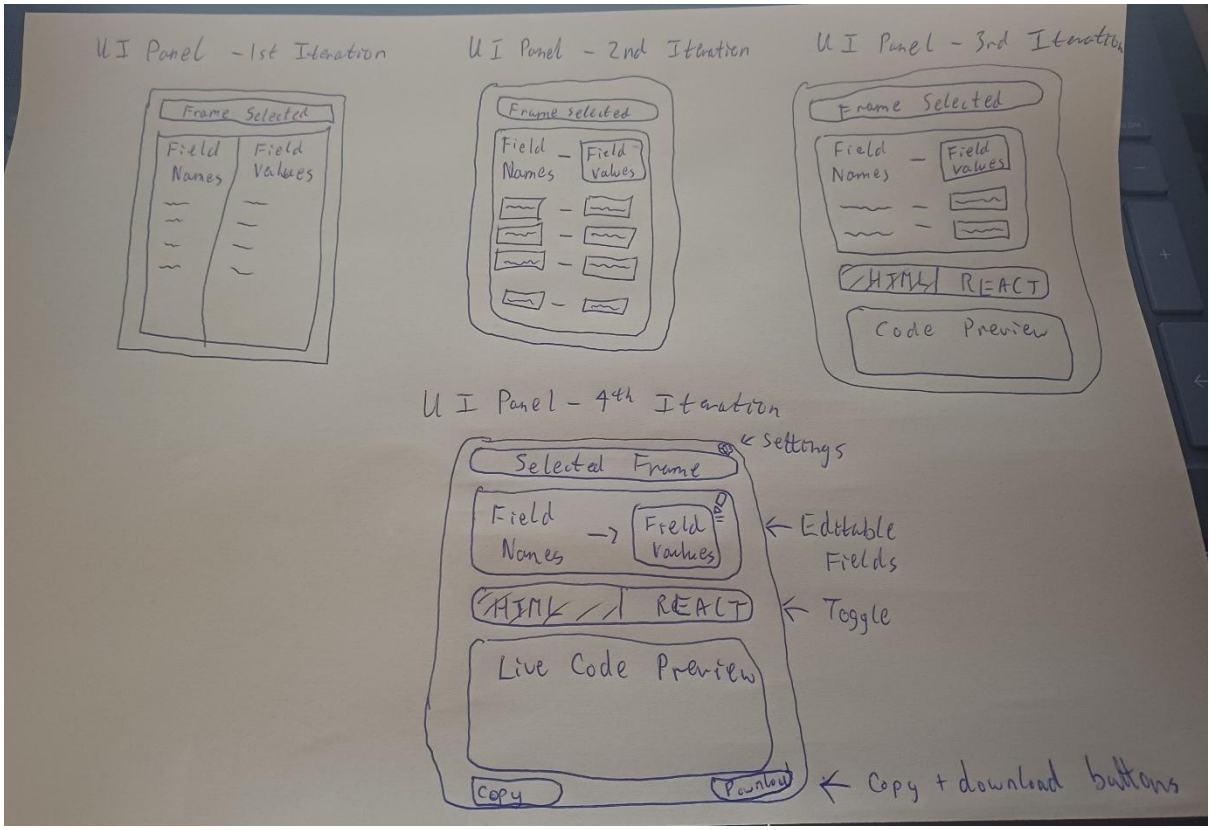


*Figure 3 - Low Fidelity Wireframes*

Wireframes were created in Figma as shown in Figure 3 to visualise and explore design options for the UI panel and to clarify feedback in an interactive environment. Attention was directed towards the code preview sections later for consideration of multiple outputs to compare HTML to React outputs. An in-app interface was crucial for reducing the need for external sources as discovered through investigation of current established plugins, which require users to outsource to multiple pages to complete a singular action. This aspect directly targets the efficiency of design to development workflows.

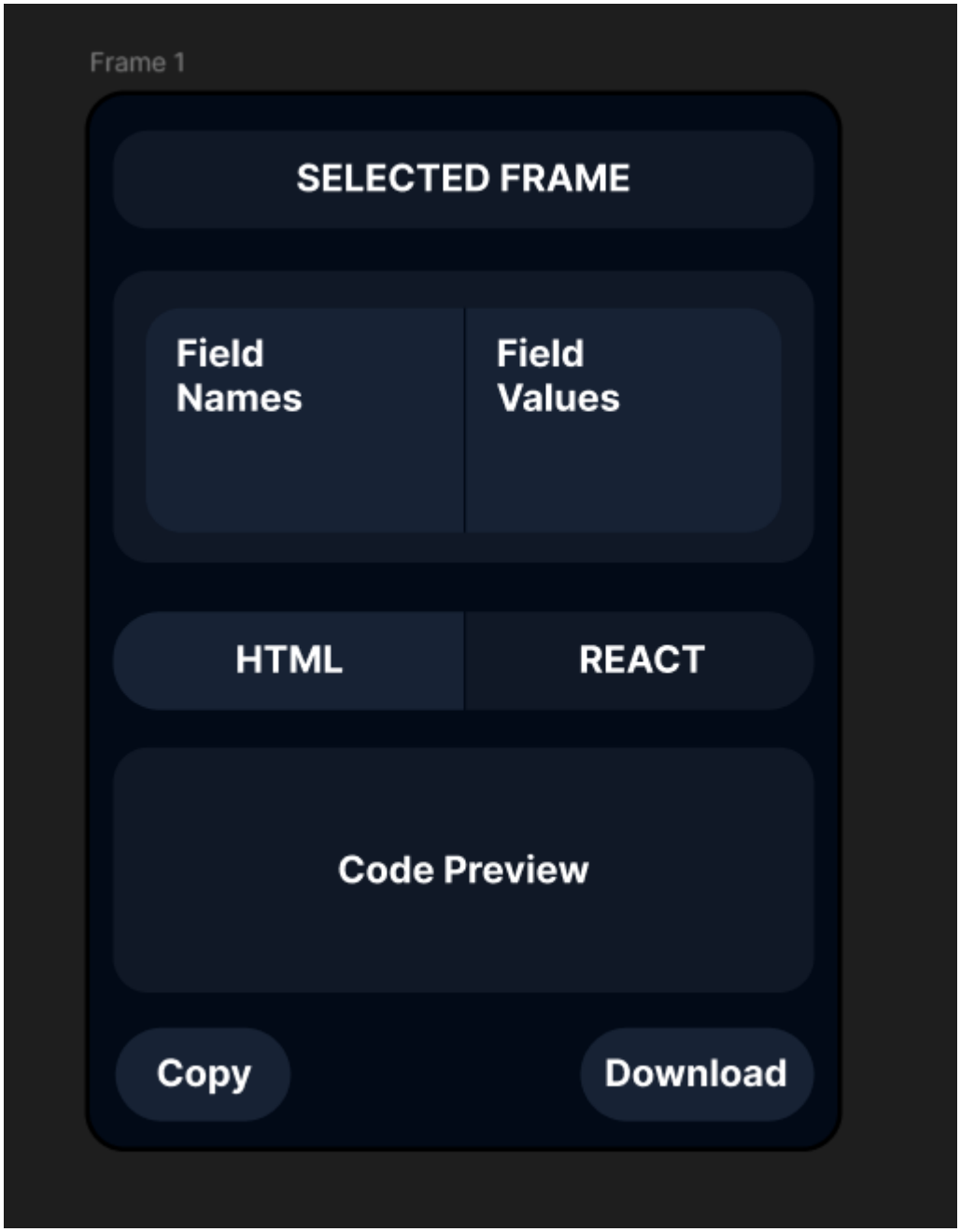


*Figure 4 - Mockup UI Panel Design*

Final Deliverable:

The final deliverable is a fully functional Figma plugin which extracts node properties from selected frames and maps the property values to Tailwind utility classes, which further is generated into HTML and React components that are structured and scaled accordingly to Tailwind values and layout schemes.

Developers are allowed to inspect, edit and preview extracted designs all in the Figma application, reducing the implementation process and chance of inconsistent design to

code conversion. The tool focuses on developers to increase component-based frontend development workflows.